

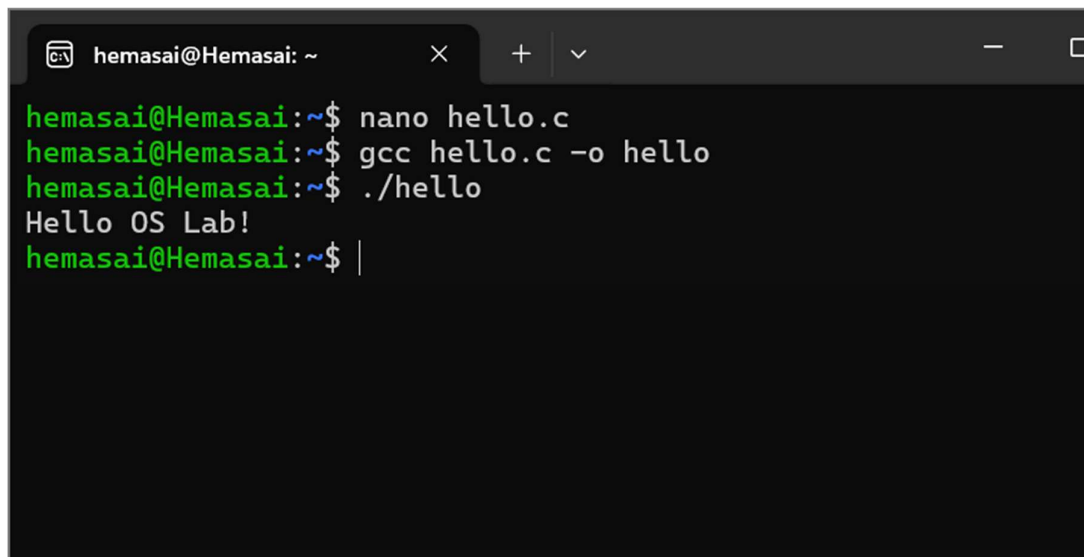
1. Hello.c

```
#include<stdio.h>
```

```
int main()
{
    printf("Hello Linux\n");
    return 0;
}
```

Compile: gcc hello.c -o hello

./hello

A terminal window with a dark background and light green text. The window title is 'hemasai@Hemasai: ~'. The terminal shows the following commands and output: 'nano hello.c', 'gcc hello.c -o hello', './hello', and 'Hello OS Lab!'. The prompt 'hemasai@Hemasai:~\$' is visible at the end of each line.

```
hemasai@Hemasai:~$ nano hello.c
hemasai@Hemasai:~$ gcc hello.c -o hello
hemasai@Hemasai:~$ ./hello
Hello OS Lab!
hemasai@Hemasai:~$ |
```

2. Read User Name

```
#include<stdio.h>
```

```
int main()
{
    char name[30];

    printf("Enter your name: ");
    scanf("%s",name);

    printf("Welcome %s\n",name);

    return 0;
}
```

```
hemasai@Hemasai: ~  
hemasai@Hemasai:~$ nano hello.c  
hemasai@Hemasai:~$ gcc hello.c -o hello  
hemasai@Hemasai:~$ ./hello  
Hello OS Lab!  
hemasai@Hemasai:~$ nano username.c  
hemasai@Hemasai:~$ gcc username.c -o username  
hemasai@Hemasai:~$ ./username  
Enter your name: hemasai  
Welcome hemasai  
hemasai@Hemasai:~$ |
```

3. Display Process ID

```
#include<stdio.h>  
#include<unistd.h>  
  
int main()  
{  
    printf("Process ID = %d\n",getpid());  
  
    return 0;  
}
```

```
hemasai@Hemasai:~$ nano pid.c  
hemasai@Hemasai:~$ gcc pid.c -o pid  
hemasai@Hemasai:~$ ./pid  
Process ID = 654  
hemasai@Hemasai:~$ |
```

4. Fork()

```
#include<stdio.h>  
#include<unistd.h>
```

```
int main()
{
    fork();

    printf("Hello\n");

    return 0;
}
```



```
Process ID = 634
hemasai@Hemasai:~$ nano fork.c
hemasai@Hemasai:~$ gcc fork.c -o fork
hemasai@Hemasai:~$ ./fork
Hello
Hello
hemasai@Hemasai:~$ |
```

5. Parent and Child

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("I am Child Process\n");
    }
    else
    {
        printf("I am Parent Process\n");
    }

    return 0;
}
```

```
}
```

```
Hello
hemasai@Hemasai:~$ nano parentchild.c
hemasai@Hemasai:~$ gcc parentchild.c -o parentchild
hemasai@Hemasai:~$ ./parentchild
I am Parent Process
I am Child Process
hemasai@Hemasai:~$ |
```

6. Display parent and Child PID

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child PID = %d\n",getpid());
        printf("Parent PID = %d\n",getppid());
    }
    else
    {
        printf("Parent PID = %d\n",getpid());
    }

    return 0;
}
```

```
I am Child Process
hemasai@Hemasai:~$ nano piddisplay.c
hemasai@Hemasai:~$ gcc piddisplay.c -o piddisplay
hemasai@Hemasai:~$ ./piddisplay
Parent PID = 800
Child PID = 801
Parent PID = 388
hemasai@Hemasai:~$ |
```

7. Wait()

```
#include<stdio.h>
```

```

#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

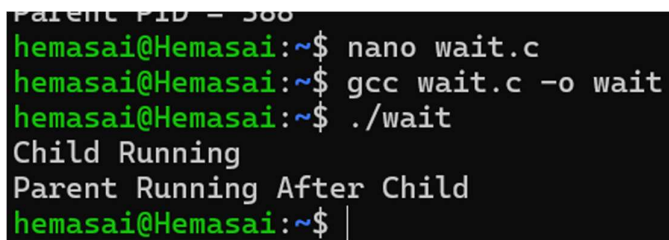
    pid=fork();

    if(pid==0)
    {
        printf("Child Running\n");
    }
    else
    {
        wait(NULL);

        printf("Parent Running After Child\n");
    }

    return 0;
}

```



```

Parent PID = 588
hemasai@Hemasai:~$ nano wait.c
hemasai@Hemasai:~$ gcc wait.c -o wait
hemasai@Hemasai:~$ ./wait
Child Running
Parent Running After Child
hemasai@Hemasai:~$ |

```

8. Wait()

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls","ls",NULL);

    printf("After exec()\n");
}

```

```
return 0;
```

```
Parent Running in child
hemasai@Hemasai:~$ nano exec.c
hemasai@Hemasai:~$ gcc exec.c -o exec
hemasai@Hemasai:~$ ./exec
Before exec()
Documents  file2.txt      hemasaifile      piddisplay.c
Hemasai    filename.c  hemasaifile.c    script.sh
OSLab      fork        hemasaiprograms  shortcut
a.txt      fork.c      new.txt          username
b.txt      hello      parentchild      username.c
data.txt   hello.c     parentchild.c    wait
exec       hello1      pid              wait.c
exec.c     hello1.c    pid.c
file1.txt  hemasai    piddisplay
hemasai@Hemasai:~$ |
```

```
}
```

9. fork() + exec()

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        execlp("date","date",NULL);
    }
    else
    {
        wait(NULL);

        printf("Child Completed\n");
    }

    return 0;
```

}

```
File1.txt  hemasai  piddisplay
hemasai@Hemasai:~$ nano forkexec.c
hemasai@Hemasai:~$ nano forkexec.c
hemasai@Hemasai:~$ gcc forkexec.c -o forkexec
hemasai@Hemasai:~$ ./forkexec
Wed Aug  5 04:01:30 UTC 2026
Child Completed
hemasai@Hemasai:~$ |
```